

CS 486/686

Dissecting a Small Language Model

Yuntian Deng

Lecture 21

One real Qwen turn, end to end

Search ›

Uncertainty ›

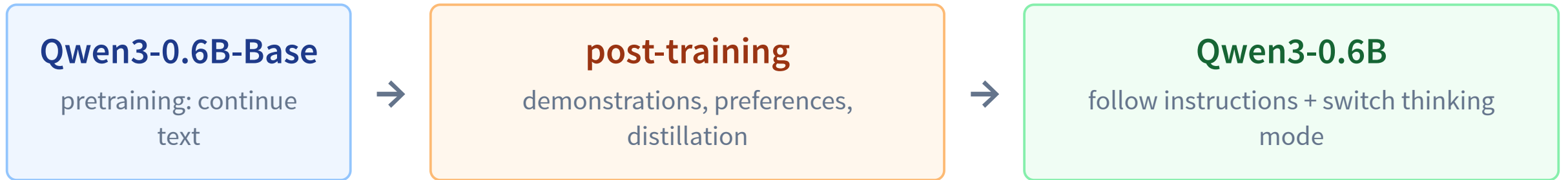
Decisions ›

Learning

By the end, you can dissect one Qwen turn

- Run **Qwen3-0.6B** in Python or a browser.
- Trace messages → template → tokens → hidden states → logits.
- Map real Qwen config fields onto a transformer block.
- Explain **prefill, decode, and the KV cache**.
- Predict how decoding and prompts change behavior.
- Recognize where a fixed model is insufficient.

L20 built a base model. Today we run an assistant.



Same causal architecture; new behavior learned after pretraining.

Post-training turns a continuer into an answerer

Same prompt to both models: What is gradient descent?

base model (pretraining only)

How does it work? What is the role of the learning rate? How does the learning rate affect the convergence of the algorithm? What is the difference between gradient descent and stochastic gradient descent? ...

continues with more question-like text

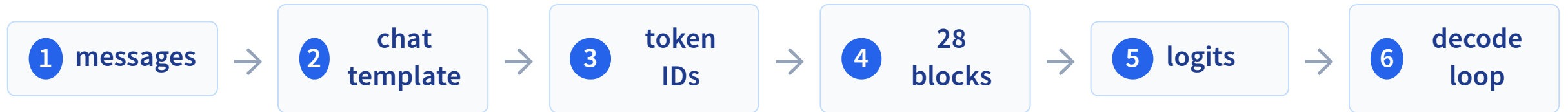
post-trained assistant

Gradient descent is a numerical method used in optimization to find the minimum (or maximum) of a function. It is commonly used in machine learning ...

answers the question

Real Qwen3-0.6B-Base vs Qwen3-0.6B. Answering is *learned* from curated (instruction, answer) demonstrations and preference data.

One user turn, boundary by boundary



We will inspect the real object at every boundary.

The real model we will dissect

Qwen3-0.6B

dense, decoder-only, post-trained

28

layers

1024

hidden width

3072

FFN width

16 / 8

query / KV heads*

128

head width

151,936

vocabulary size

32K

max training context

tied

input/output matrix

* grouped-query attention — explained in a few slides. Qwen Team, “Qwen3 Technical Report,” 2025.

Config is architecture written as data

```
{  
  "hidden_size": 1024,  
  "intermediate_size": 3072,  
  "num_hidden_layers": 28,  
  "num_attention_heads": 16,  
  "num_key_value_heads": 8,  
  "head_dim": 128,  
  "vocab_size": 151936,  
  "tie_word_embeddings": true  
}
```

state _ residual stream

communicate 16 query heads share 8 K/V heads

compute 3072-wide SwiGLU sublayer

predict 151,936 logits; input/output weights tied

The API receives structured messages

```
messages = [  
    {  
        "role": "user",  
        "content": "Explain gradient descent in one sentence"  
    }  
]
```

role

who is speaking?

content

what did they say?

The model still consumes tokens—so this structure must become one string.

The chat template serializes one turn

`<|im_start|>`

user

Explain gradient descent in one sentence.

`<|im_end|>`

`<|im_start|>`

assistant

generation starts here

`<|im_start|>/<|im_end|>` are ChatML message-boundary tokens
(**im** = input message); each is one special token in the vocabulary.

No system message appears unless we actually supplied one.

Inspect the exact serialized input LIVE DATA

Single turn or multi-turn → Qwen special tokens → thinking ON/OFF generation cue.

single turn

reasoning

future fact

multi-turn

thinking: OFF

structured message

user Explain gradient descent in one sentence.

exact string sent to tokenizer

```
<|im_start|>user
Explain gradient descent in one sentence.
<|im_end|>
<|im_start|>assistant
<think>

</think>
```

20 tokens after serialization

Template pre-fills an empty <think></think>; Qwen starts the answer.

Blue marks chat-control tokens. Green marks the thinking protocol. The user content is unchanged.

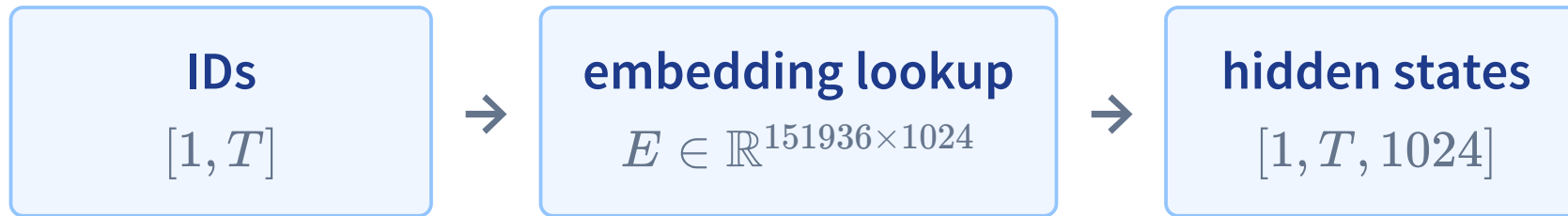
Tokenization happens after templating



... role markers + generation cue

· means a leading space. IDs shown are actual Qwen tokenizer outputs, not illustrative.

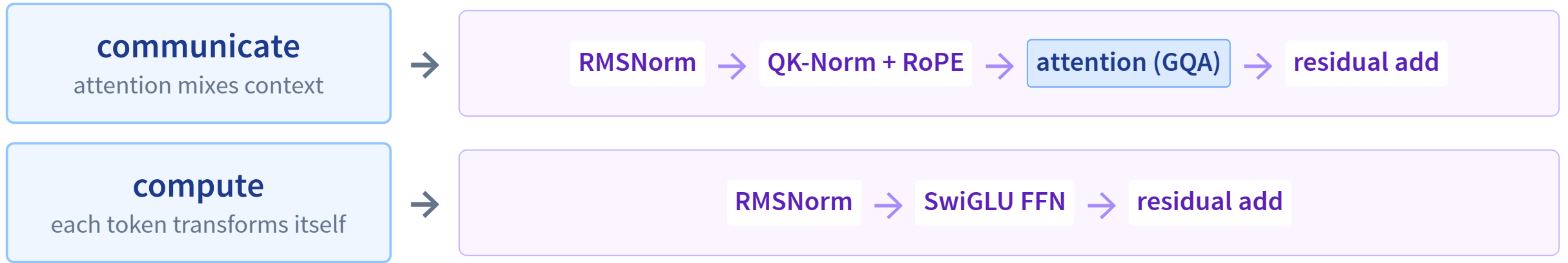
Token IDs become a $[T, 1024]$ residual stream



Qwen position: RoPE rotates query/key dimensions inside attention; it is not an added position vector.

The same matrix E returns at the end to score output tokens.

L19's block, with Qwen's real component names



The abstraction is unchanged; these names specify Qwen's implementation.

Grouped-query attention: 16 Q heads share 8 K/V heads



Why share? Keep many query views, but store half as many K/V heads in the cache.

The real model repeats this pattern: 16 query heads grouped over 8 key/value heads.

The same residual stream passes through 28 blocks



Every block adds an update; the residual stream carries the evolving $[T, 1024]$ representation.

Guess where “it” looks, then check

REAL MODEL DATA

Guess, reveal the measured row, then switch sentences: the same layer/head behaves consistently.

"The robot picked up th..."

"The dog chased the cat..."

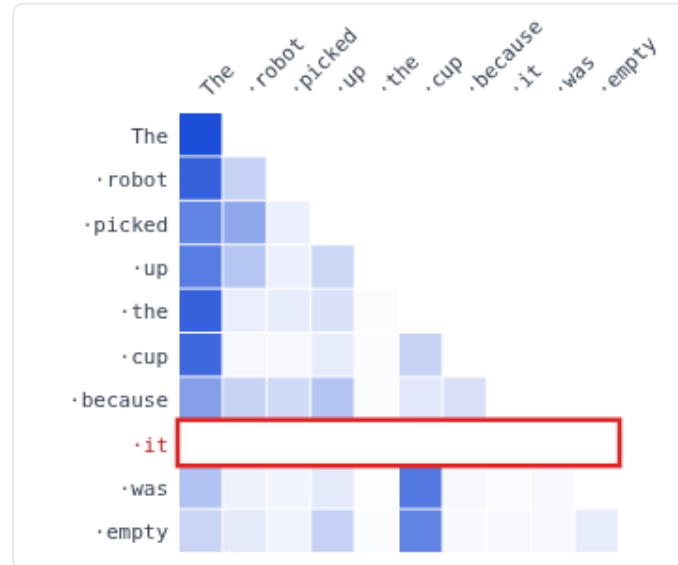
coreference: it → robot

previous-token pattern

broad-context pattern

layer 11, head 1 · largest measured attention from query 'it' to a content token

The · robot · picked · up · the · cup · because · **·it** · was · empty



query: ·it attends to...

Guess: does **it** point to the nearest noun (**cup**)?

reveal measured row

Same layer/head across two sentences: the coreference head links "it" to the earlier noun both times. Rows sum to ~1 with zero future mass; a curated head is a measurement, not a full explanation.

Attention is a measurement—not an explanation

one head

among 16 heads × 28 layers

curated

selected because its pattern was
interpretable

not causal proof

large weight does not prove decisive
influence

Use attention to inspect a computation—not to claim the model’s full reason.

Jain & Wallace, “Attention is not Explanation,” NAACL 2019; Wiegrefe & Pinter, “Attention is not not Explanation,” EMNLP 2019.

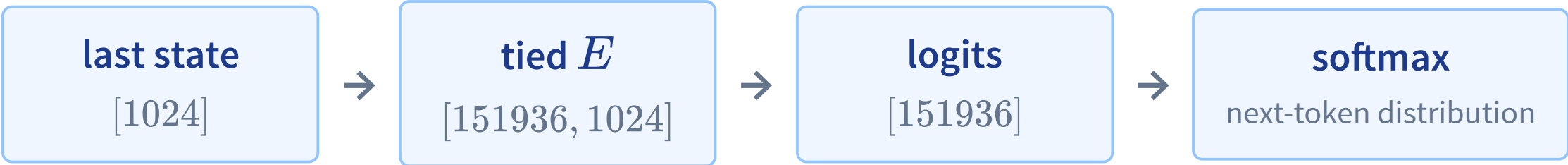
One matrix does input and output



$$151,936 \times 1024 \approx \text{156M parameters} (\sim 0.3 \text{ GB})$$

that is a quarter of a 0.6B model, so Qwen **shares one matrix** instead of two.

The final state is scored by the tied embedding matrix



Gradient		0.985
The		0.004
A		0.004
all other tokens		~0.007

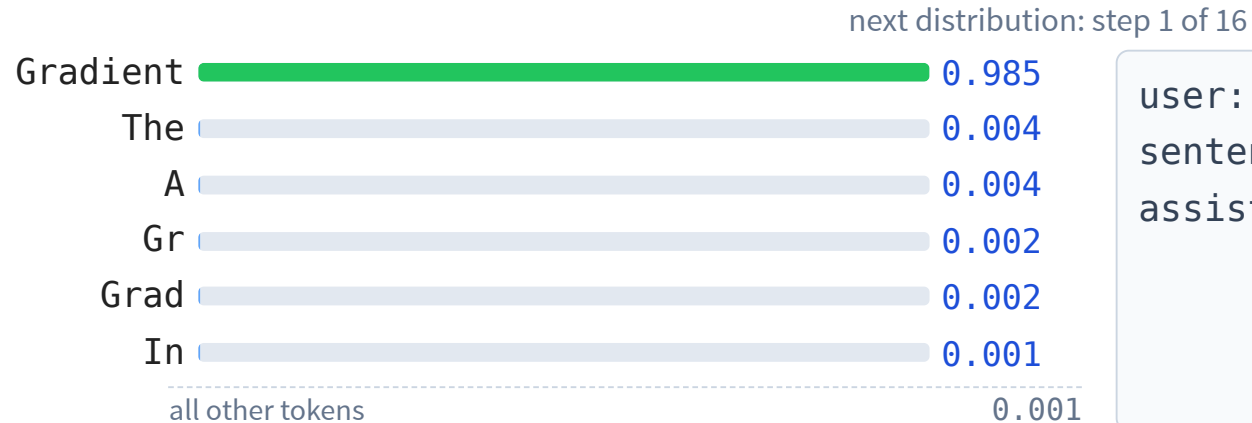
Real Qwen trace after the non-thinking template for “Explain gradient descent in one sentence.”

Append one token, then score again

REAL MODEL DATA

Step through an exact Qwen continuation; every distribution is conditioned on the enlarged context.

Qwen3-0.6B post-trained assistant · non-thinking prompt · greedy trace.



user: Explain gradient descent in one sentence.
assistant:

append next token

reset

Candidate labels use · for a leading space. Every click reads a newly conditioned distribution.

Inference has two phases

1. prefill

prompt tokens in parallel

Build hidden states and K/V cache for the whole prompt.

2. decode

one new token

Generate sequentially, one distribution per step.

Parallel prefill; sequential decode.

Prefill: the prompt is already known

Every prompt token of Explain gradient descent in one sentence. is available up front.

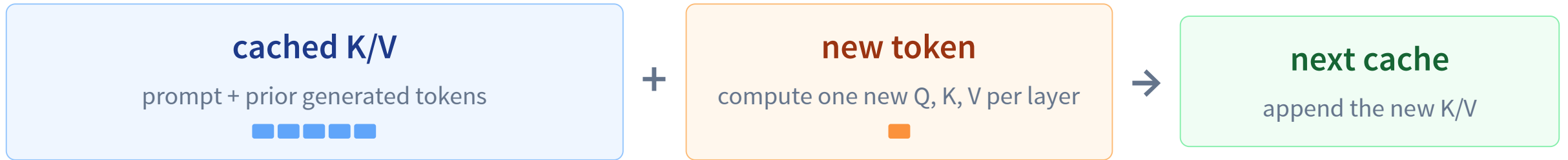
<|im_start|> user Explain ·gradient ·descent ... assistant

one parallel forward pass $\rightarrow$ hidden states + K/V for all positions

first next-token distribution: Gradient ≈ 0.985

Prefill = the single forward pass over the known prompt we traced earlier.

KV cache reuses the past at every layer



The new query still attends over every cached source;
caching avoids recomputing old K/V and hidden states.

Causality freezes the past, so we reuse it

already computed

Gradient ·descent ·is ·a

hidden states + K/V never change (each token only sees earlier tokens)

new token

·method

only this column is computed this step

Generating token t reuses all $t - 1$ cached K/V; it recomputes nothing behind it.

How much recomputation does the cache avoid?

LIVE

Step through prefill and decode; compare token-work with and without cached K/V.

Prefill processes all four prompt tokens and builds four K/V slots.

The robot picked up the

cached K/V slots



4 token positions computed without cache

4 token positions computed with cache

decode next token

reset

For three output tokens: without cache $4 + 5 + 6 = 15$ token positions; with cache $4 + 1 + 1 = 6$.

Context is working memory—and cache memory

prompt **generate** free window

window

prompt + output share Qwen's 32k
context

cache memory

grows with $\text{layers} \times \text{KV heads} \times \text{length}$

latency

decode remains token-by-token

The cache buys speed by spending memory.

The complete Python inference path

1. load tokenizer + post-trained weights

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")
```

```
model = AutoModelForCausalLM.from_pretrained(  
    "Qwen/Qwen3-0.6B", torch_dtype="auto", device_map="auto")
```

2. serialize the role/content messages

```
text = tok.apply_chat_template(messages, tokenize=False,  
    add_generation_prompt=True, enable_thinking=False)
```

3. tokenize and move tensors beside the model

```
inputs = tok(text, return_tensors="pt").to(model.device)
```

4. autoregressively append new token IDs

```
all_ids = model.generate(**inputs, max_new_tokens=64,  
    do_sample=True, temperature=.7, top_p=.8, top_k=20)
```

5. remove the prompt IDs, then decode only the answer

```
new_ids = all_ids[0, inputs.input_ids.shape[1]:]
```

```
answer = tok.decode(new_ids, skip_special_tokens=True)
```

real greedy output for this prompt:

```
# "Gradient descent is a method used to minimize a function
```

```
# by iteratively adjusting the parameters ... negative gradient."
```

The same path can run in the browser

```
import { pipeline } from "@huggingface/transformers";

const qwen = await pipeline(
  "text-generation",
  "onnx-community/Qwen3-0.6B-ONNX",
  { device: "webgpu", dtype: "q4f16" },
);

const messages = [{
  role: "user",
  content: "Explain gradient descent.",
}];

const result = await qwen(messages, {
  max_new_tokens: 64,
  do_sample: true, temperature: 0.7, top_p: 0.8, top_k: 20,
});
```

instant path

audited traces embedded in the deck

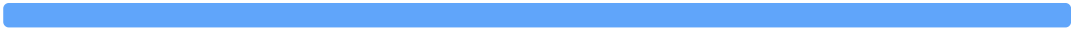
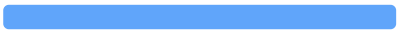
optional live path

several-hundred-MB download +
compatible WebGPU

The concepts and default demos never depend on the live download succeeding.

One distribution, two selection rules

Qwen context: Write a creative name for a friendly blue robot.

**		0.559	greedy always choose argmax: **
Sure		0.205	
Here		0.059	sampling draw according to probability: varied path
Maybe		0.052	

The weights and distribution are fixed; only the selection rule changes.

Temperature reshapes; top-k and top-p trim

temperature

: sharper : unchanged : flatter

Ranking stays the same.

top-k = 3

keep **, Sure, Here

Remove every token below rank 3.

top-p = .8

$.559 + .205 + .059 = .823$

Keep the smallest prefix reaching 0.8.

renormalize

sample only from the kept candidates

Run Qwen with recommended decoding presets

REAL MODEL DATA

Prompt Explain gradient descent in one sentence.
— each preset has its own exact distribution and continuation.

greedy

recommended non-thinking

recommended thinking

argmax

thinking OFF

Gradient		0.985
The		0.004
A		0.004
Gr		0.002
Grad		0.002
In		0.001

full distribution; choose argmax

user: Explain gradient descent in one sentence.
assistant:

append next token

reset

optional: run live

Precomputed controls are exact. Candidate labels use · for a leading space; thinking traces begin with <think>.

Longer generation: a math word problem

REAL MODEL DATA

Thinking mode reasons for many tokens before answering — step through it or reveal the full generation.

next distribution: step 1 of 16



user: Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did she sell altogether in April and May?
assistant:

append next token

reset

show full answer

optional: run live

Thinking mode writes a long <think> trace before the answer. Append tokens, or reveal the full generation.

Prompting changes tokens in context—not weights

role

You are a patient tutor.

task

Explain gradient descent.

audience

Use language for a 10-year-old.

examples

Input/output pairs specify a pattern.

format

One sentence; no equations.

zero-shot + examples = few-shot prompt

general prompt

“Gradient descent is a method used to minimize a function ...”

10-year-old prompt

“Gradient descent is like a way to get closer to the best answer ...”

Real greedy Qwen outputs. More instructions and examples are simply more conditioning tokens.

Thinking mode changes the generation protocol

thinking OFF

<think>

</think>

Template pre-fills an empty block, then Qwen answers.

$$T = .7, p = .8, k = 20$$

thinking ON

<|im_start|>assistant

...

Qwen generates **<think>...</think>** before its answer.

$$T = .6, p = .95, k = 20$$

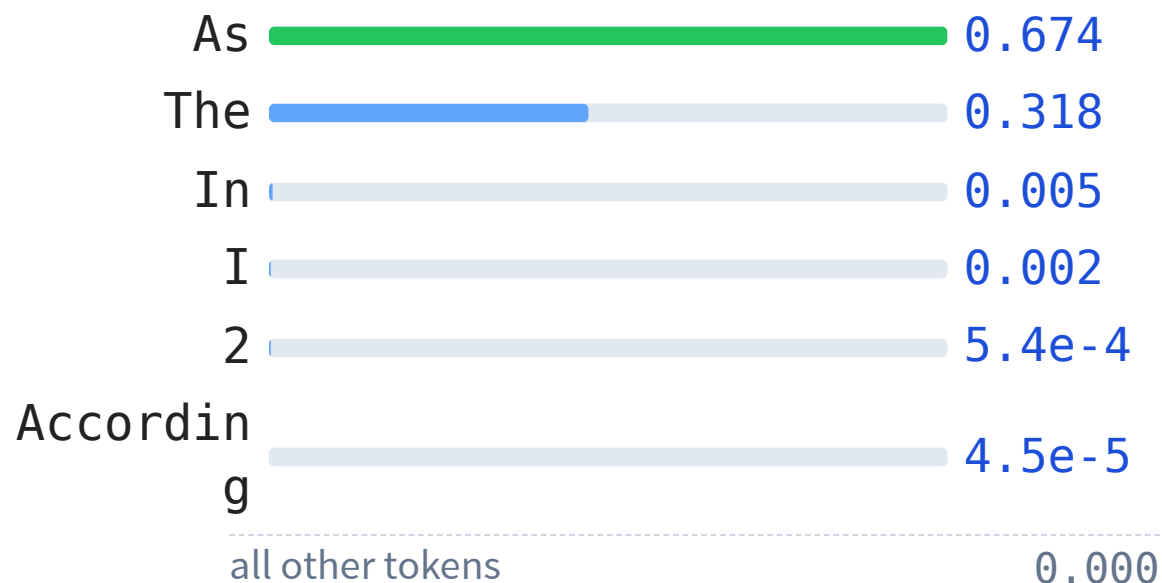
Reasoning text can help on hard tasks, but costs tokens and is not guaranteed correct or causally faithful.

A fluent answer can still be fabricated

REAL MODEL DATA

Ask about the future and watch a confident, invented answer generate token by token.

next distribution: step 1 of 16



user: Who won the 2031 Turing
Award?
assistant:

append next token

reset

optional: run live

The model has no post-2024 knowledge, yet it still produces a confident, fabricated name.

What changed today—and what did not

changed today

chat template, context, decoding rule

did not change

weights, stored knowledge, available actions

We steered a fixed model through its inputs and its decoding—nothing more.

Recap

- Messages $\rightarrow$ chat template $\rightarrow$ exact tokens $\rightarrow [T, 1024]$ states.
- Qwen block: RMSNorm, RoPE + QK-Norm, grouped-query attention, SwiGLU.
- Tied E scores $\sim 152k$ logits; softmax gives the next-token distribution.
- Prefill in parallel, decode sequentially, reuse the KV cache.
- Decoding and prompts steer behavior; the weights stay fixed.

L22: when inputs are not enough—prompting vs RAG/tools vs SFT/LoRA, and Qwen as a frozen interpreter for PAW.